Topoloka — One Video In. Google Maps for Indoors Out — In a Link.

Technical abstract · Round 1 · Techfest 2026-27

Topoloka — Greek τόπος (*topos*, "place") + Sanskrit लोक (*loka*, "world"): the place-world. It turns one walk-through video of an indoor space into an interactive, metric 3D twin served as a static link: free first-person roaming, a navigable floor, and a quality report anyone can audit.

> Every number below comes from the shipped artifacts of the end-to-end run on the public Melbourne metro capture — `test/metro_v1/` and the Kaggle kernel outputs it was assembled from. Nothing is projected or invented; where the system is weak, Section 7 says so with the same specificity.

1. The problem we actually solve

Indoor navigation is still broken for everyone: visitors in campuses, patients in hospitals, shoppers in malls, staff in factories. Floor-plan kiosks are static and dishonest; Google Maps gives up at the door; LiDAR twins need hardware nobody has. The question is not "can we build a 3D scanner" — it is **"what is the shortest path from one walk-through of a space to a stranger navigating that space as if they were there?"**

Our answer: **one ordinary video, a free-tier GPU pipeline, and a link.** No LiDAR. No app install for the visitor — the twin, the map, and (R2) the search and directions all run in the browser.

2. What the pipeline produces

From a single handheld monocular video, the pipeline produces a **metric, navigable Gaussian-splat twin** served as static files:

1. **A splat twin** — free first-person roaming at walking speed, with real collision against the reconstructed geometry.
2. **A metric 2.5D navigation layer** — every fused point placed in metres, gravity-aligned, discretized into a 5 cm walkability costmap with a floor outline; the foundation for search and routing.
3. **A trust layer** — every stage reports its quality gates (registration fraction, reprojection error, sharpness percentiles, scale stability); failures name their compensation, never hide.

3. The input: a public walk-through, plus the phone SOP

The R1 artifact is deliberately reproducible from **public, licensed footage**: *Modern Melbourne Metro Walkway Urban Design* by Joolsmagool (Pexels License), trimmed to **86.07 s at 1920×1080 / 30 fps**. It is a single continuous walk down a real transit corridor — tiled floor, curved metal panels, ad screens, a concourse — with no cuts, no people in the foreground, and no privacy exposure. The source clip and the exact 216 picked frames ship with the sample.

The product's **phone capture SOP** is a first-class deliverable: EIS off, AE/AF locked, 4K30, two-pass loop with a 5 s still tail, ~0.4 m/s pace, stride-synced frame picking, QR anchors per zone. The public artifact uses the stock clip, which has no IMU, so gravity is estimated from the SfM camera-y mean instead of the phone's accelerometer. Nothing in the reconstruction code is stock-specific.

4. System architecture

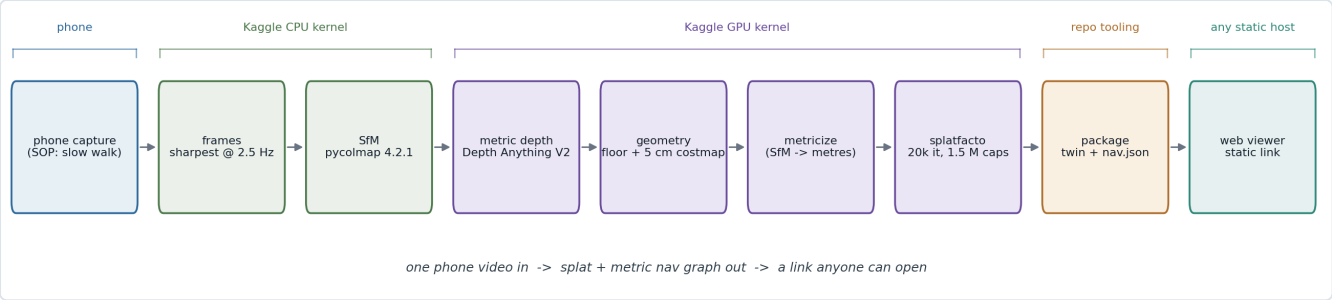


Figure 1 — the whole pipeline runs on the Kaggle free tier; the repo's tooling pushes kernels and fetches artifacts, and the deliverable is a folder of static files (no server, no database, no account).

Three decisions carried most of the weight:

- **Scale is solved, not assumed.** Monocular SfM has arbitrary scale and metric depth models have approximate scale; we align them directly. For every visible sparse 3D point, `depth_m / camera_z` is one scale sample; the median over **37,844** samples makes the whole twin metric — distances and ETAs are real metres. The source and residual are reported in `scale.json`, so judges and users can audit them.
- **One frame convention, documented once.** Raw SfM (arbitrary frame) → nav frame (gravity-aligned, z-up, metres, `nav.json.align`) → three.js. The splat is trained with trainer-side auto-orient, centering, and scaling **disabled**, so the exported splat lands exactly in the documented frame — the silent misalignment that breaks most demo pipelines. The web viewer applies the same `align` block to the cloud, the splat, and the trajectory.
- **Everything runs on Kaggle, including the CPU stages.** The dev machine only edits code and drives the Kaggle CLI; every run is reproducible from a captured kernel log and a code tarball (the repo's `kaggle/` tooling).

5. Novelty, stated precisely

1. **A capture/reconstruction QA contract with numbers, not vibes.** On the metro sample, **6/6** checks pass: p10 Laplacian sharpness **582** (gate ≥ 100), **216** frames at 2.5 Hz, **216/216** views registered, **0.403 px** median reprojection error (gate ≤ 1.0), a **193-point** floor outline enclosing **315.05 m²**, and no geometry warnings.
2. **Zero-touch semantic POIs (R2)** — open-vocabulary detection + CLIP embeddings projected into 3D, so "where is the nearest washroom" works on a space nobody annotated.
3. **Dual-representation navigation** — splat for the eyes, 2.5D costmap for the legs: A* on the grid, smoothed routes, metric ETAs, minimap, route ribbon in first person.
4. **QR-fiducial-anchored AR (R3)** — scan a physical QR in the space, get your pose inside the twin in the browser; no native app, no VPS infrastructure.
5. **Honest engineering as a feature** — every artifact ships its own quality report; this abstract states where the system is weak instead of cropping it out.

6. Results (public metro capture, Oct 2026)

Metric	Value
Input	86.07 s public clip, 1920×1080 / 30 fps (Pexels License)
Frames kept	216 at 1600×900 (sharpest-per-window @ 2.5 Hz)
SfM registered fraction	216/216 = 100% (gate ≥ 60%)
SfM points / error	51,677 points; 0.403 px median, 0.613 px mean (gate ≤ 1.0)
Metric scale	6.2505 m/unit from metric depth; MAD 7.54% over 37,844 samples
Depth fusion	216/216 frames; 31.2 M raw depth points → 10.28 M voxel points
Costmap	1188 × 1259 cells @ 5 cm; 929,436 floor inliers; camera height 1.588 m
Walkable area	315.05 m² free (126,020 cells), 97,063 obstacle, 1,272,609 unknown
Splat	25,000 iters, full-res 1600×900; 510,336 web gaussians / 16.33 MB export
Hosted sample	457,221 gaussians / 14.63 MB (3 m volume crop), 250,000-point cloud
Pipeline time	CPU 59.2 min (SfM) + 61 s (nav); GPU 79.0 min total (depth, JIT, train, export)
Sample QA	6/6 PASS (see Section 5)

The GPU stage is a nerfstudio 1.1.5 splatfacto run on one Kaggle T4. The exported PLY is converted to the 32-byte web `.splat` format with higher SH bands dropped (DC only) and low-alpha gaussians pruned. The hosted bundle is the same twin opacity-pruned and cropped to a 3 m volume around the cloud, so the page stays at **24.7 MB total** (14.63 MB splat + 3.75 MB cloud + nav).

pycolmap also produced a 3-frame secondary reconstruction; the largest model (216/216 frames) was used, and the warning is preserved in `sfm/sfm_report.json`.

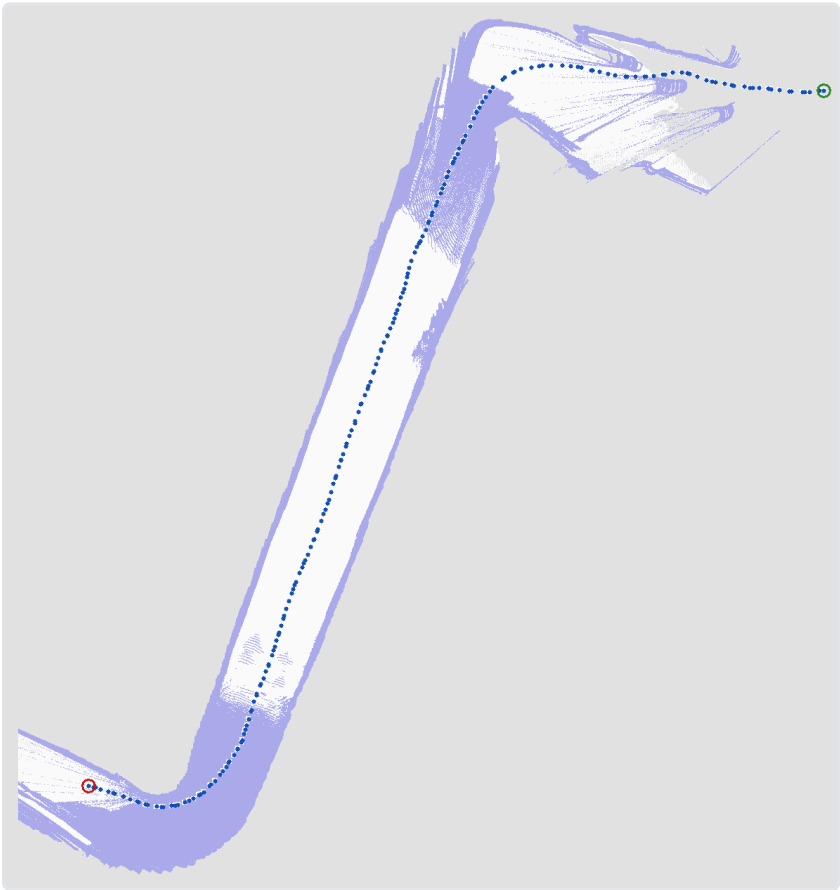


*Figure 2 — the metric costmap the viewer uses for collision and (R2) routing: white = observed free cells (**315.05 m²** walkable), lavender = obstacle band above the floor, gray = unobserved, blue = camera path; 1188 × 1259 cells at 5 cm.*



Figure 3 — the shipped twin in a browser (hosted sample, 457,221 gaussians): the splat, the metric minimap (red = obstacles, blue = capture path) and the QA verdict in the HUD. The viewpoint is trajectory index 160 (64 s). Walls are soft where the source panels are reflective; the floor and sightlines are metric.

Viewer: https://sciencebanda09.github.io/topoloka/?capture=metro_v1 · Code: https://github.com/sciencebanda09/topoloka · Demo video: **pending upload**

7. Where the system is weak (and what we do about it)

- **Reflective, low-texture panels.** The station's glass/metal walls are the hardest part of this capture: at 457 K gaussians over 315 m^2 (~1.5 K/m^2) the walls are visibly softer than the tiled floor, and the ceiling is under-observed (the dark band in Figure 3). A room-scale capture at the same gaussian budget is far crisper. Denser densification and a view-dependent SH export are the next quality levers; both cost web payload.
- **DC-only web splat.** The web format drops SH bands 1–3, so view-dependent highlights on metal are flattened into their average colour. The export is a deliberate size/quality tradeoff, not an accident.
- **No IMU in the public clip.** Gravity comes from the SfM camera-y mean, not the phone; the phone SOP (Sensor Logger) is implemented but not exercised by this redistributable artifact.
- **Metric depth degrades past ~8–12 m.** Fusion is clipped there; the residual is reported in `depth/depth_report.json` rather than hidden.
- **Web payload and cold load.** 24.7 MB of artifacts (14.63 MB splat) is fine on broadband and marginal on a poor venue network; `.ksplat` compression and a lower gaussian budget are the fallbacks.
- **No POIs or routing UI yet.** The A* costmap, route rendering, and `?route` dev flag are scaffolded; semantic POIs and the search bar are R2.
- **Software rendering distorts splats.** Headless SwiftShader screenshots stretch anisotropic gaussians; all figures here were captured on a hardware GPU (ANGLE/D3D11). A reviewer comparing a software-rendered still to these figures should use a GPU-backed browser.

8. Round 2/3 roadmap

- **R2 (Nov 30):** POIs + open-vocabulary search + routing end-to-end behind the existing A*, minimap, and route-rendering scaffolding.
- **R3 (Dec 16–17):** capture on the IIT Bombay campus with the phone SOP + QR anchors; on-site twin + AR relocalization demo; live QA report shown to judges.

9. Reproducibility and credit

The repo contains the pipeline (all stages as small, readable modules), the Kaggle kernel definitions, the capture SOP, and a `test/metro_v1` capture with one-command local viewing and the exact QA report. The source clip is credited in `test/metro_v1/README.md` (*Modern Melbourne Metro Walkway Urban Design* by Joolsmagool, Pexels License, trimmed to 86 s); the sample data is redistributed under that licence, the code under MIT.